

3.1.1 Lists (Ordered, Mutable, Methods)

فہرست (List) مربع بریکٹ [] میں لکھی جاتی ہے، ترتیب برقرار رکھتی ہے اور تبدیل کی جا سکتی ہے (Mutable).

CS XII

Chapter 3: Programming Fundamentals

Lecture 02 of 18

★ Golden

3.11 Lists (Ordered, Mutable, Methods) 4.5 min

1 minute 7
A list is Python's most versatile built-in data structure. It stores an **ordered** collection of items in a single variable. Lists are **mutable** (you can change them after you create them) and they are written with **square brackets** `[]`.
Why use a list? One variable can hold thousands of values. You can mix types if you need to. You can add, remove and sort items as the program runs.

```
numbers = [16, 2, 15, 14, 3, 20] # list of integers
names = ["Banana", "Apple", "Mango"] # list of strings
mixed = [1, "Python", 3.14, True] # mixed data types
print(numbers)
print(names)
print(mixed)
```

```
names = ["apple", "mango", "banana", "grape", "kiwi"]
["Banana", "Apple", "Mango"]
["Python", 3.14, True]
```

List indexing (positive and negative)

Each item sits in a numbered slot called an **index**. Numbering starts at 0 from the left. Python also supports **negative indexing** from the right: `-1` is the last item, `-2` is the second last, and so on.



```
fruits = ["apple", "mango", "banana", "grape", "kiwi"]
print(fruits[0]) # First item
print(fruits[2]) # Last item (when you keep the length)
print(fruits[-1]) # Last item (works even if the list grows)
print(fruits[-2]) # First item, counted from the right
```

```
apple
mango
banana
grape
kiwi
```

REMEMBER IT
Always remember `[-1]`, if a list has 10,000 items and you do not want to compute `len(list) - 1`, write `list[-1]`. It grabs the last item in one step.

Slicing a list
Slicing creates a portion of a list into a **new list**. The original is not changed.

Syntax: `list[start:stop:step]`
start — first index to include (default 0)
stop — first index to exclude. The slice goes up to, but does **not** include, this index.
step — jump size (default 0). A step of `-1` walks backwards.

```
nums = [10, 1, 2, 3, 4, 5, 6, 7, 8, 9]
print(nums[1])
print(nums[4])
print(nums[2])
print(nums[:3])
```

```
nums = [10, 1, 2, 3, 4, 5]
print(nums[1])
print(nums[4])
print(nums[2])
print(nums[:3])
```

Trace of `nums[2:4]`: start at index 2 (value 2), take 3, 4, 5, and stop **before** index 6. Result `[2, 3, 4, 5]`. `nums[::-1]` is the standard way to reverse a list without changing the original (it returns a new reversed list).

Traversals
Traversals means visiting each element, one by one, in order. Loop a `for` loop (the usual Python style) or a `while` loop with an index.

```
numbers = [10, 20, 30]
for num in numbers:
    print(num)
```

```
num = 10
20
30
```

```
numbers = [10, 20, 30]
index = 0
while index < len(numbers):
    print(numbers[index])
    index += 1
```

```
10
20
30
```

The `for` loop gives you each value directly. The `while` loop needs a counter, a test `index < len(numbers)`, and `index += 1` so the loop ends.

Built-in list methods

Because a list is mutable, Python gives you methods that change it in place. `len()` is listed with them in many notes, but it is a **built-in function**: you write `len(fruits)`, not `fruits.len()`.

Operator	What it does
<code>len(list)</code>	Returns the number of items.
<code>.append(value)</code>	Adds an item at the end.
<code>.insert(index, value)</code>	Inserts an item at a chosen position (later items shift right).
<code>.remove(value)</code>	Removes the first matching value. Raises <code>ValueError</code> if it is missing.
<code>.clear()</code>	Removes every item. The list becomes <code>[]</code> .
<code>.count(value)</code>	How many times a value appears.
<code>.index(value)</code>	Index of the first occurrence.
<code>.sort()</code>	Sorts the list in place, ascending. Use <code>reverse=True</code> for descending.
<code>.reverse()</code>	Reverses the list in place (does not sort).

WORKING EXAMPLE

Worked example: methods change the same list step by step.

```
fruits = ["apple", "banana", "apple"]
print(fruits)
fruits.append("grape")
print(fruits)
fruits.remove("apple")
print(fruits)
fruits.sort()
print(fruits)
```

```
fruits = ["apple", "banana", "apple"]
fruits.append("grape")
fruits.remove("apple")
fruits.sort()
print(fruits)
```

After this line	list (memory)
<code>fruits</code>	<code>['apple', 'banana', 'apple']</code>
<code>.append("grape")</code>	<code>['apple', 'banana', 'apple', 'grape']</code>
<code>.remove("apple")</code>	<code>['banana', 'apple', 'grape']</code> — only the first apple goes
<code>.sort()</code>	<code>['apple', 'banana', 'grape']</code>

More methods on numbers:

```
nums = [10, 20, 30]
print(len(nums))
nums.insert(1, 40)
print(nums)
print(nums.index(20))
print(nums.index(40))
nums.reverse()
print(nums)
nums.clear()
print(nums)
```

```
nums = [10, 20, 30]
print(len(nums))
print(nums)
print(nums)
```

`insert(1, 40)` places 40 at index 1, so 20 and 30 shift right. After that, 30 sits at index 3. `reverse()` flips the current order; it does not sort. `clear()` leaves an empty list `[]`, not the value `None`.

You can also replace an item by index, because lists are mutable.

```
numbers = [0, 9, 40]
numbers[1] = 99
print(numbers)
```

```
numbers = [0, 9, 40]
```

Advantages of lists

- Mutable** — add, remove or replace items as the program runs.
- Organized** — thousands of values live in one variable instead of `val1, val2, val3, ...`
- Flexible** — integers, strings, booleans and even other lists can all sit in the same list.

Activity: searching an element in a list

The program continues a list, user input, a `for` loop, an `if` test and a **flag** is Boolean that remembers whether the search succeeded.

```
numbers = [5, 10, 25, 30, 35]
search = int(input("Enter number to search: "))
found = False
for num in numbers:
    if num == search:
        found = True
        break
print("Number found in the list.")
if not found:
    print("Number NOT found in the list.")
```

Execution 1 — the user types 15, which is at index 2. The loop sets `found = True` and `break` stops the scan at once.

```
found = True
Enter number to search: 15
Number found in the list.
```

Execution 2 — the user types 99. The loop finishes without a match, so the flag stays `False`.

```
found = False
Enter number to search: 99
Number NOT found in the list.
```

Why `break`? If the value is found at index 1 in a list of 10,000 items, there is no need to check the remaining 9,999. `break` makes linear search cheaper as soon as it succeeds.

CONSEQUENCE IT
List — Lists of items, indexed from 0, Square brackets, **Mutable** (mutable), **Size** memory trick, **Start** is 0, **Stop** is out

COMMON MISTAKE

Mistake 1: `.remove("apple")` deletes every apple. **Fix:** It deletes only the first match. Call it again if you need the next one, or use a loop.

Mistake 2: `.sort()` and `.sorted()` are the same. **Fix:** `.sort()` is a list method that changes the original and returns `None`. `.sorted()` is a function that works on many structures and returns a new list (see 3.13).

Mistake 3: `fruits[1]` on a five-item list. Valid positive indices are 0 to 4. Use `[-1]` for the last item.

آردو میں سمجھیں
فہرست (list) دو چیزکتا 11 میں لکھی جاتی ہے: ترتیب و فراز رکھنے پر اور تبدیلی کی جا سکتی ہے (Mutable). اندیشہ ہمارے میں شروع ہوتا ہے اور 1-1 آخری عنصر دیتا ہے. Append، آخر میں جوڑا ہے. Remove، پہلے ملنا، فہرست ہٹا دیتا ہے اور [-1] اصل فہرست کو ترتیب دیتا ہے۔ لکھی گئی وقت break مل جاتی ہے۔ ہم یہاں یہ دیکھ رہے ہیں۔

CLEAR TIP

Lists appear every year: "What's a list", list four methods with one line each, "Differentiate list and tuple", "Write a program to search an element", "Predict the output of a slice". Write ordered, mutable, `[-1]` index from 0, `[-2]` for last. For a slice, always say the stop index is excluded. For `.remove()`, say "first occurrence only". **Flag** a short program ready that uses `append`, `remove` and `sort` with the printed list after each line.

Check yourself

1. What does `nums[2:4]` return if `nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]`?

2. After `fruits = ["apple", "banana", "apple"]` then `fruits.remove("apple")`, what is `fruits`?

3. Why does the search program use `break`?

Answers: 1. `[2, 3, 4, 5]`, 2. `["banana", "apple"]` (only the first apple is removed), 3. To stop searching once the value is found, so the rest of a long list is not checked.

Next lecture: 3.1.2 Tuples (Ordered, Immutable)